Name: Mimanshu Gahlaut

UID: 24BAI70038

Course: BE-CSE (AI&ML)

Subject: Database Management System

Experiment 10: Trigger

1. Aim of the Session

To design a trigger that automatically implements the functionality of a primary key, ensuring unique identification of records without manual intervention.

2. Software Requirements:

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database

- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

3. Objective of the Session

To create a database trigger that automatically enforces primary key constraints or generates unique key values, replicating the functionality of a stored procedure.

4. Practical / Experiment Steps

The work was carried out through the following activities:

1. **Table Structure Design:** Created the employees table with appropriate attributes, defining emp_id as the primary key to uniquely identify each record.
2. **Sequence Creation:** Defined a database sequence (emp_seq) to generate a continuous series of unique values for automatic ID assignment.
3. **Trigger Function Development:** Implemented a PL/pgSQL function to check whether a primary key value is provided during insertion.

4. **Automatic Key Assignment Logic:** Incorporated logic within the function to fetch the next sequence value using nextval() when emp_id is not supplied.
5. **Trigger Configuration:** Linked the function to the table using a BEFORE INSERT trigger to ensure execution prior to data insertion.
6. **Encapsulation of Logic:** Ensured that the key generation process is handled internally within the database, eliminating the need for manual intervention.
7. **Testing and Validation:** Inserted multiple records without specifying emp_id and verified the results using a SELECT query to confirm correct sequential ID generation.

5. Procedure of the Practical

Execution was performed in the following order:

- Opened PostgreSQL and started a new database session.
- Created the employees table using the CREATE TABLE command.
- Defined a sequence starting from 1 for ID generation.
- Wrote the auto_emp_id() trigger function for dynamic key assignment.
- Implemented logic to assign a value only when emp_id is not provided.
- Created the trigger trg_auto_emp_id and attached it to the table.
- Inserted sample records without specifying the primary key.
- Executed a SELECT * query to confirm that IDs were generated automatically in sequence (1, 2, etc.).
- Recorded the results to show how triggers can simulate auto-increment behavior used in industry systems.

6. I/O Analysis (Input / Output Analysis)

Input Queries

SQL

```
CREATE TABLE employees1 (
    emp_id INTEGER PRIMARY KEY,
    emp_name VARCHAR(100),
    department VARCHAR(50)
);
```

Query | Query History

```sql
CREATE TABLE employees1 (
    emp_id INTEGER PRIMARY KEY,
    emp_name VARCHAR(100),
    department VARCHAR(50)
);
```

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 187 msec.
```

```sql
CREATE SEQUENCE emp_seq
START 1;
```

```sql
CREATE SEQUENCE emp_seq
START 1;
```

Data Output | Messages | Notifications

```
CREATE SEQUENCE

Query returned successfully in 118 msec.
```

```sql
CREATE OR REPLACE FUNCTION auto_emp_id()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.emp_id IS NULL THEN
        NEW.emp_id := nextval('emp_seq');
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```sql
CREATE OR REPLACE FUNCTION auto_emp_id()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.emp_id IS NULL THEN
        NEW.emp_id := nextval('emp_seq');
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Data Output | Messages | Notifications

```
CREATE FUNCTION

Query returned successfully in 230 msec.
```

```
CREATE TRIGGER trg_auto_emp_id
BEFORE INSERT ON employees1
FOR EACH ROW
EXECUTE FUNCTION auto_emp_id();
```

```
20  CREATE TRIGGER trg_auto_emp_id
21  BEFORE INSERT ON employees
22  FOR EACH ROW
23  EXECUTE FUNCTION auto_emp_id();
```

Data Output | Messages | Notifications

```
CREATE TRIGGER

Query returned successfully in 115 msec.
```

```
INSERT INTO employees1 (emp_name, department)
VALUES ('AliceD', 'HR');

INSERT INTO employees1 (emp_name, department)
VALUES ('Bob', 'IT');
```

```
25  INSERT INTO employees1 (emp_name, department)
26  VALUES ('AliceD', 'HR');
27
28  INSERT INTO employees1 (emp_name, department)
29  VALUES ('Bob', 'IT');
```

Data Output | Messages | Notifications

```
INSERT 0 1

Query returned successfully in 119 msec.
```

```
SELECT * FROM employees1;
```

```
31  SELECT * FROM employees1;
```

Data Output | Messages | Notifications

SQL

	emp_id [PK] integer	emp_name character varying (100)	department character varying (50)
1	1	Alice	HR
2	2	Bob	IT

7. Learning Outcome

- **Automation using Triggers:** Learned how to eliminate manual primary key entry through trigger-based automation.
- **Use of Sequences:** Gained understanding of generating unique identifiers using database sequences.
- **Maintaining Data Integrity:** Learned how BEFORE INSERT triggers help validate and modify data before storage.
- **Encapsulation of Logic:** Understood how databases can independently manage key assignment without relying on application-level code.